REVIEW 2

Visual Cognitive Coding Assistant with Multilingual Learning Support

Project Category: RESEARCH

Guide Name: Dr. Navneet Nayan
Designation: Assistant Professor
Department: CINTEL

Student Names & Registration Numbers

Aastha Hotwani (RA2311026010389)

Priyanshu Kumar Singh (RA2311033010088)

Abstract

Students often struggle in coding because they don’t understand where they are going wrong.
Most AI tools just give answers, but they don’t help students improve their thinking.
Visual Cognitive Coding Assistant solves this problem by focusing on how students think while coding.

It combines multiple smart features into one system:

- **Error Pattern Detection** — Finds repeated mistakes in logic
- **Predictive Guidance** — Suggests possible future errors before they happen
- **Visual Learning Support** — Shows flowcharts and step-by-step logic
- **Multilingual Support** — Explains concepts in simple languages like Hindi/Tamil
- **Progress Tracking** — Monitors improvement over time

Instead of just giving solutions, the system helps students learn, think better, and solve problems efficiently.

Introduction

Many students face difficulty in coding because they repeat the same mistakes and do not understand their logic.
Current tools:

- Give direct answers
- Do not track learning
- Do not analyze thinking process

Because of this:

- Students don’t improve effectively
- Learning becomes slow and frustrating

There is a need for a system that:

- Identifies mistakes
- Explains logic clearly
- Helps students improve step by step

Student improvement calculating formula

$$Score = (1 - ErrorRate) + Improvement$$

Literature Survey

S.No	Paper / Authors (Year)	Methodology	Key Contribution	Limitation
1	Aggarwal et al. (2024) – *LLMs in Programming Contests*	Evaluation of GPT-4 & Claude using pass@1 metrics	Empirical analysis of LLM performance in contests	No cognitive learning analysis
2	Liang et al. (2024) – *Performance Study of LLM Code*	Runtime validation using LeetCode API	Studies correctness & execution behavior	No student behavior tracking
3	Xie et al. (2024) – *Execution-Free Runtime Detection*	Static analysis + REDO framework	Detects runtime errors without execution	No adaptive feedback
4	Li et al. (2024) – *LLM Code Mistakes Analysis*	ReAct prompting with GPT-4	Categorizes syntax & logic mistakes	No personalization
5	Pei et al. (2025) – *AP2O*	Progressive LLM fine-tuning	Improves AI code correction accuracy	Focus on model, not learner
6	Wang et al. (2023)	Self-correcting AI coding loop	Reduced coding errors	No visual reasoning support
7	Kumar & Patel (2021)	Explainable AI tutoring systems	Improved learner engagement	No algorithm visualization
8	Olson et al. (2023) – *AI Code Completion Survey*	Survey-based evaluation	Analyzes impact of AI tools on learning	No predictive modeling

S.No	Paper / Authors (Year)	Methodology	Key Contribution	Limitation
9	Zhang et al. (2024) – *FlowCE Benchmark*	Multi-dimensional flowchart evaluation	Evaluates flowchart reasoning abilities	No student progress tracking
10	Feng et al.	OCR + Image segmentation + GenAI	Extracts logical representation from flowcharts	No adaptive tutoring
11	Chen et al. – *Flow2Code*	Flowchart-to-code benchmarking	Evaluates image-to-code conversion	No cognitive modeling
12	Garcia et al. (2022)	Multimodal vision-language models	Improved multimodal reasoning	Limited educational focus
13	Ahmed & Roy (2020)	UML diagram parsing	Accurate UML extraction	Documentation-only focus
14	Singh et al. (2024)	AI programming assistants	Improved beginner learning	No behavior prediction
15	Dey et al.	ML-based competitive skill assessment	Industry placement readiness modeling	No real-time feedback system
16	Brown et al. (2022)	LLM-based code generation	High code correctness	Text-only, no cognitive tracking

Research Gaps

- Most AI coding tools generate final solutions but do not analyze the student's thinking process.
- Existing systems lack personalized coding profiles based on individual behavior and mistake patterns.
- Current error detection methods identify mistakes but do not predict repeated or future errors.
- Most platforms provide one-time feedback without tracking long-term learning progress.
- There is no integrated system combining cognitive analysis, visual learning, predictive feedback, and multilingual support for competitive programming education.

Research Objectives

EPIC 1: Cognitive Code Analysis Engine

(Core intelligence of system)

User Stories

- As a student, I want to submit partial or incorrect code so that I can receive feedback without being given the full solution.
- As a student, I want the system to analyze my logic flow so that I can understand where my thinking went wrong.
- As a student, I want guided hints instead of direct answers so that I can improve my problem-solving skills.

EPIC 2: Personalized Code Profiling (Code DNA)

(Behavior tracking + learning personalization)

User Stories

- As a student, I want the system to track my coding patterns so that I can understand my strengths and weaknesses.
- As a student, I want a personalized profile showing my coding style so that I can improve consistently.
- As a student, I want insights like loop usage, condition handling, and structure complexity so that I can refine my coding approach.

Research Objectives

EPIC 3: Error Pattern Memory System

(Learning from past mistakes)

User Stories

- As a student, I want the system to remember my past mistakes so that I don’t repeat them.
- As a student, I want alerts when I make similar errors again so that I can correct them faster.
- As a student, I want feedback based on my previous submissions so that my learning becomes continuous.

EPIC 4: Visualization & Learning Intelligence

(biggest differentiator — visual learning)

User Stories

- As a student, I want visual representations of my algorithm so that I can better understand how it works.
- As a student, I want to see time complexity comparisons so that I can optimize my solutions.
- As a student, I want progress graphs over time so that I can track my improvement.

Research Objectives

EPIC 5: Multilingual Learning Support

(Accessibility + inclusivity)

User Stories

- As a student, I want explanations in my preferred language so that I can understand concepts better.
- As a student, I want code feedback translated into simple language so tht complex ideas are easier to grasp.
- As a student, I want to switch languages dynamically so that I can learn comfortably.

EPIC 6: Adaptive Feedback & Recommendation System

(AI-driven improvement loop)

User Stories

- As a student, I want personalized recommendations for problems so that I can improve weak areas.
- As a student, I want difficulty levels to adapt based on my performance so that learning stays challenging but achievable.
- As a student, I want suggestions for optimization techniques so that I can write better code.

Research Objectives

EPIC 7: Student Progress Tracking Dashboard

(Analytics layer for long-term learning)

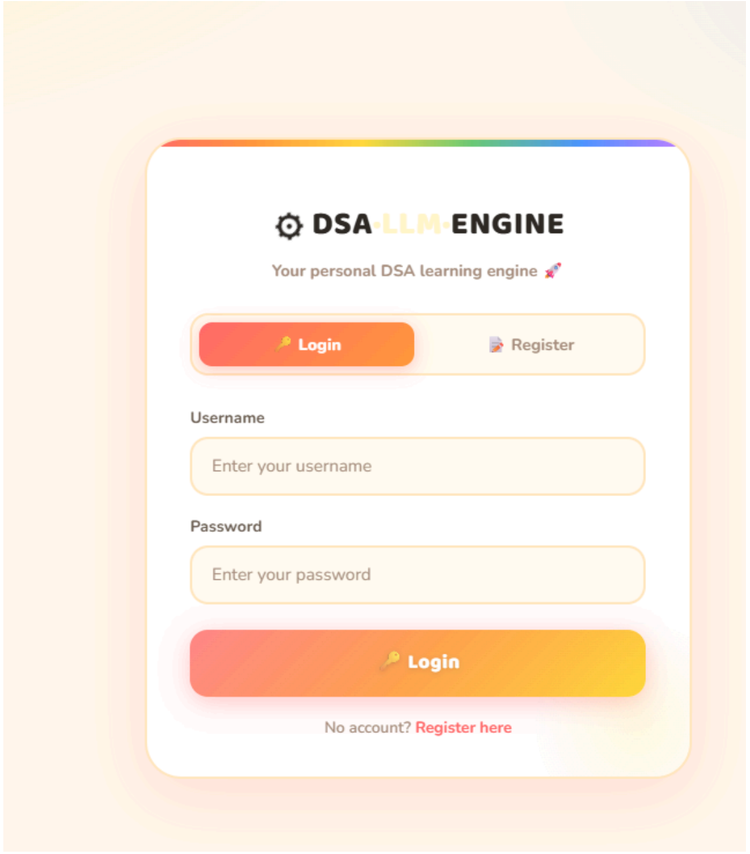
User Stories

- As a student, I want a dashboard showing my progress so that I can monitor my growth.
- As a student, I want performance metrics (accuracy, time complexity, attempts) so that I can evaluate myself.
- As a student, I want milestone achievements so that I stay motivated.

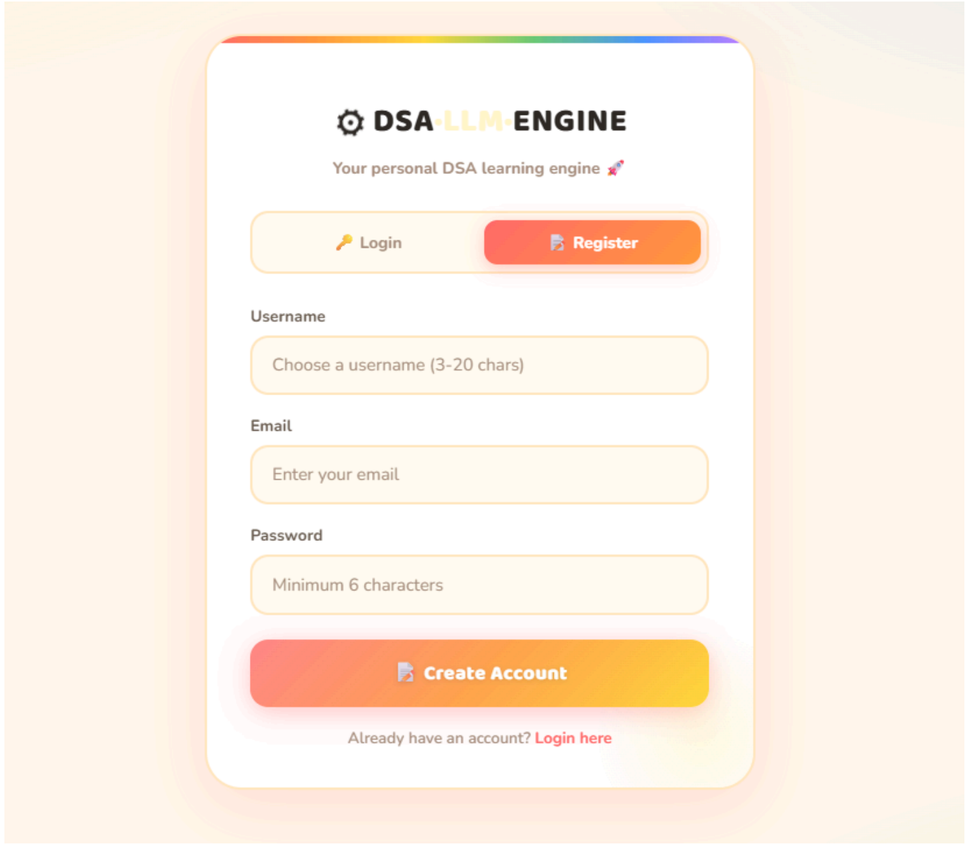
Sprint Planning

Sprint	Focus Area	Key Deliverables	Status
Sprint 1	Cognitive Code Analysis Engine	- Basic code input system, Syntax & structure parsing module, Initial logic analysis implementation	✓ Done
Sprint 2	Cognitive Code Analysis Engine	- Guided feedback generation (no full solution), Detection of logical errors, Hint-based response system	✓ Done
Sprint 3	Personalized Code Profiling (Code DNA)	- Feature extraction (loops, conditions, nesting), Pattern recognition module, Initial student profile generation	✓ Done
Sprint 4	Error Pattern Memory System	- Database for storing past errors, Similarity matching (pattern/embedding-based), Repeated mistake detection alerts	✓ Done
Sprint 5	Visualization & Learning Intelligence	- Algorithm flow visualization, Time complexity representation graphs, Optimization comparison visuals	✓ Done
Sprint 6	Multilingual Learning Support	- Multi-language explanation module, Language selection feature, Simplified explanation generation	✓ Done
Sprint 7	Adaptive Feedback & Progress Tracking Dashboard	- Personalized recommendations engine, Student performance dashboard, Progress tracking graphs & insights	✓ Done

UI Implementation — User Authentication

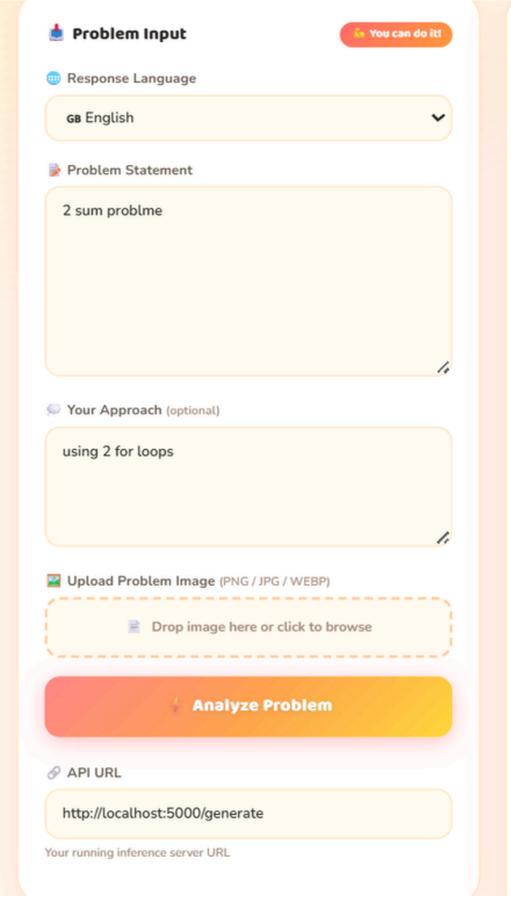


Login Page

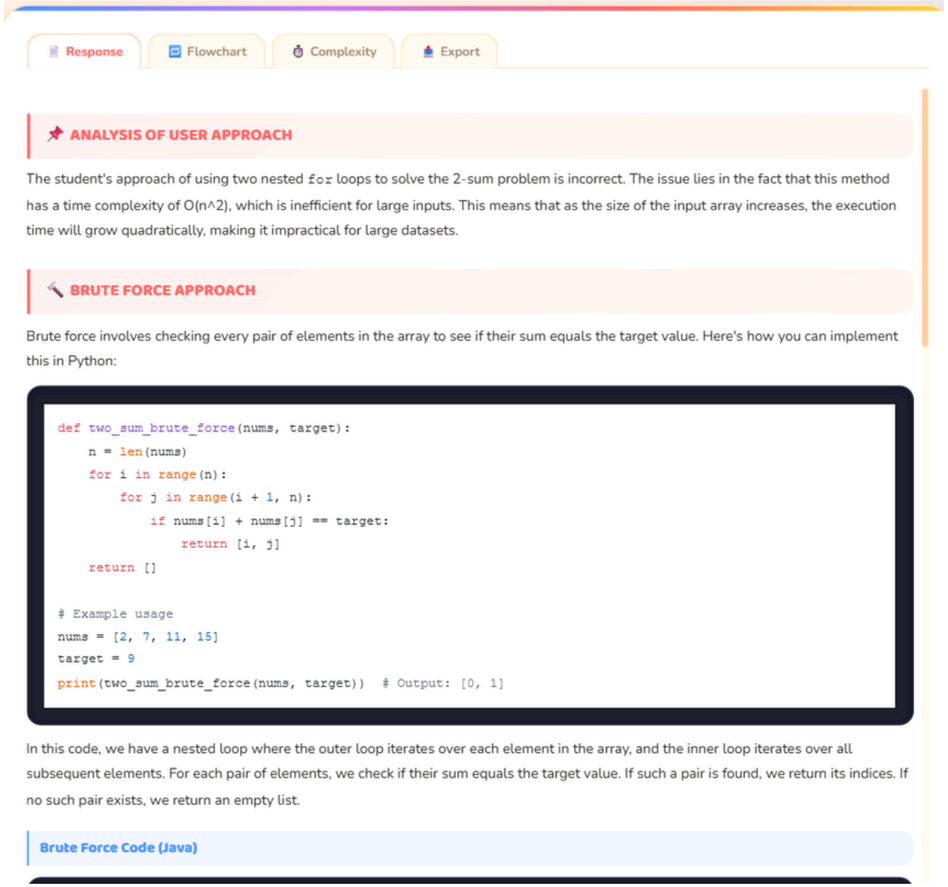


SignUp Page

UI Implementation — Problem Solving Interface

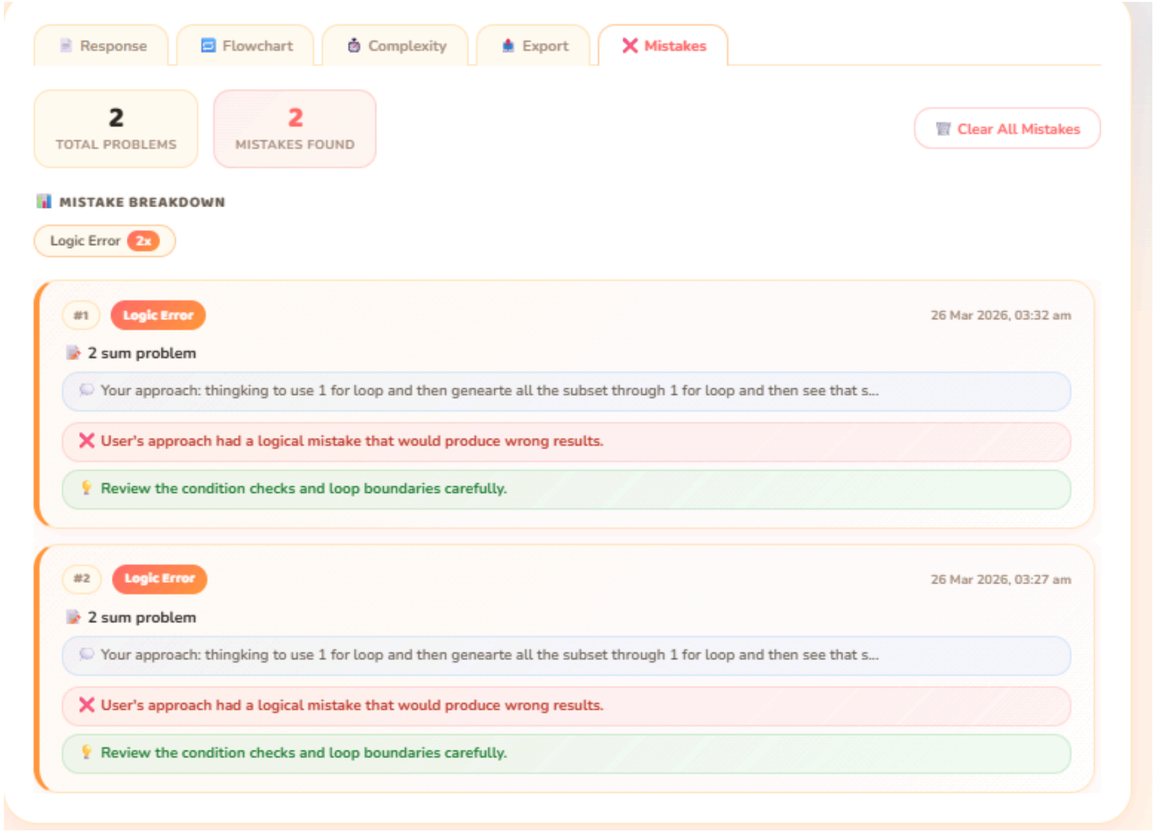


Problem Input Interface

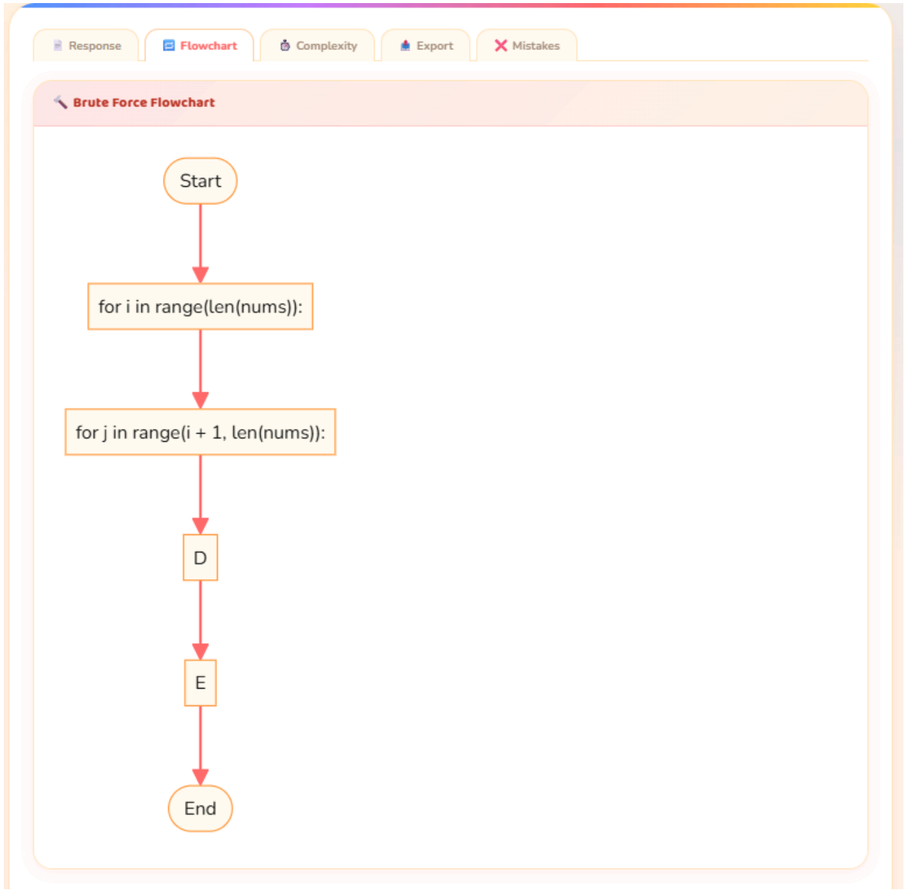


Cognitive Analysis Dashboard

UI Implementation — Error detection and Visual Learning



Error Detection & Feedback



Visual Learning (Flowchart)

Justification of Project SDG

SDG 4: Quality Education, which aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

Enhancing Quality of Technical Education

The system improves how students learn Data Structures and Algorithms by focusing on understanding mistakes and strengthening problem-solving skills rather than promoting answer copying.

Promoting Skill-Based Learning

By analyzing coding patterns and logical errors, the project helps students build real industry-relevant skills required for placements and competitive programming.

Supporting Inclusive Education

With multilingual explanations, the system makes technical learning more accessible to students from different language backgrounds, reducing language barriers in education.

Encouraging Personalized Learning

The AI-driven feedback adapts to each student’s coding behavior, making learning more student-centered and effective.

Conclusion

1. Developed a system that focuses on improving student thinking, not just giving answers
2. Successfully detects repeated mistakes and logical errors
3. Provides visual explanations to improve understanding
4. Supports multilingual learning for better accessibility
5. Helps students solve problems faster and more efficiently

References

[1] P. Aggarwal, S. Ugare, D. Contractor, and N. Modani, “Are Large Language Models a Threat to Programming Contests?,” arXiv preprint arXiv:2409.05824, 2024.
[2] Z. Liang, C. Zhang, X. Wang, and J.-G. Lou, “A Performance Study of LLM-Generated Code on LeetCode,” arXiv preprint arXiv:2407.21579, 2024.
[3] X. Xie, Y. Wu, Y. Hu, H. Yu, J. Weng, and H. Cui, “Execution-Free Runtime Error Detection for Coding Agents,” arXiv preprint arXiv:2410.09117, 2024.
[4] J. Li, K. Yao, T. Gao, and M. Zaharia, “A Deep Dive Into Large Language Model Code Generation Mistakes,” arXiv preprint arXiv:2411.01414, 2024.
[5] Y. Pei, R. Yang, H. Yu, Y. Hu, and X. Xie, “AP2O: Correcting LLM-Generated Code Errors Type by Type,” arXiv preprint arXiv:2510.02393, 2025.
[6] Anonymous, “First Multi-Dimensional Evaluation of Flowchart Understanding,” arXiv preprint arXiv:2406.10057, 2024.
[7] Y. Feng, Z. Wang, and Y. Zhang, “Parsing and Understanding Flowchart Using Generative AI,” in Proc. Int. Conf. on Artificial Intelligence, 2023.
[8] J. Chen and Anonymous, “Flow2Code: Evaluating LLMs for Flowchart-to-Code Generation,” arXiv preprint arXiv:2506.02073, 2025.
[9] S. Dey, A. Srivastava, and S. Agarwal, “Assessing Competitive Programmers for Industry Placement,” in Proc. ACM Conf. on Learning at Scale, 2023.
[10] B. M. Olson, K. Boehnke, and L. G. Ortiz, “Students’ Perspectives on AI Code Completion,” arXiv preprint arXiv:2311.00177, 2023.